Lecture Notes: Decision Trees

CIC1205 - Machine Learning

Eduardo Bezerra

March 4, 2026

0 Contents

1 Introduction **2**
1.1 Why Decision Trees Matter . . . 2

2 Tree Structure and Prediction **2**
2.1 Nodes, Edges, and Leaves . . . 2
2.2 Prediction as a Test Sequence . . . 3

3 Learning a Decision Tree **3**
3.1 The Recursive Induction Procedure . . . 3
3.2 Stopping Criteria . . . 4

4 Decision Trees for Classification **5**
4.1 Impurity and "Purity" . . . 5
4.2 Entropy and Information Gain . . . 5
4.2.1 Weighted entropy after a split . . . 6
4.3 Gini Impurity . . . 6

5 A Worked Example: "Play Golf" Split Quality **7**
5.1 Information Gain Values (Illustrative) . . . 7
5.2 Gini Computation for `Outlook` . . . 8

6 Decision Trees for Regression **9**
6.1 MSE as Node Impurity . . . 9
6.2 Overfitting in Regression Trees . . . 10

7 Inductive Bias of Decision Trees **10**

8 Practical Remarks **11**
8.1 Advantages . . . 12
8.2 Limitations . . . 12

9 Summary **12**

A Appendix: Two Quick Sanity Checks **13**
A.1 Gini for Binary Classification . . . 13
A.2 Why the Mean Minimizes Squared Error in a Leaf . . . 13

1 Introduction

A **decision tree** is a predictive model that takes an input example $\mathbf{x}$ described by a set of predictive features and their values, and returns a prediction for a target variable y.

Decision trees can be used for:

- **Classification**: y is a discrete label (e.g., spam vs. not spam).
- **Regression**: y is a real number (e.g., house price).

Decision Tree

A **decision tree** is a model that represents a prediction procedure as a **hierarchical sequence of tests** on feature values. Internal nodes implement tests (predicates); leaves store predictions.

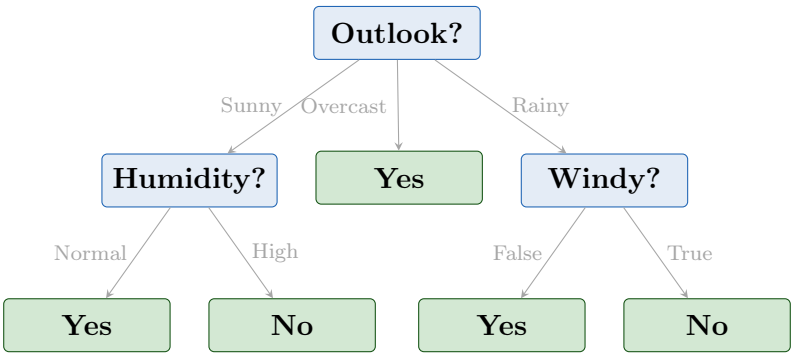

Figure 1: Example of a classification decision tree for the "Play Golf" problem. Blue nodes are **internal nodes** (feature tests); green nodes are **leaves** (class predictions). Edges carry the outcome of the test.

1.1 Why Decision Trees Matter

Decision trees are popular because they combine:

- **Interpretability**: the model can be read as a set of decision rules.
- **Flexibility**: works with numerical and categorical features.
- **Low preprocessing**: typically no need for feature scaling.

Entry Ticket 1

Give one example of a **classification** problem and one example of a **regression** problem. For each example, describe what a *single internal-node test* might look like (e.g., "feature > threshold").

2 Tree Structure and Prediction

2.1 Nodes, Edges, and Leaves

A decision tree has:

- **Internal nodes**: each is associated with a predicate involving one feature.
- **Edges/branches**: correspond to the outcomes of the predicate.
- **Leaf nodes**: store the output (a class label / class distribution, or a numeric value).

2.2 Prediction as a Test Sequence

To predict for a new example $\mathbf{x}$:

1. Start at the **root**.
2. Evaluate the predicate at the current node.
3. Follow the branch consistent with $\mathbf{x}$.
4. Stop at a leaf and output its stored prediction.

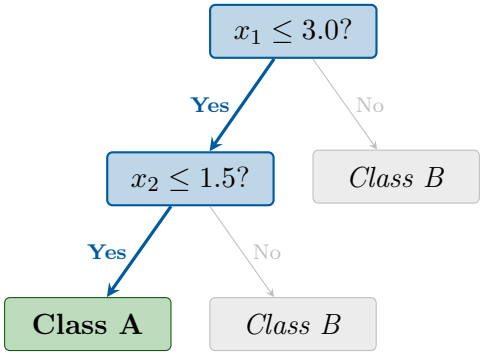

Figure 2: Tracing a prediction for an example with $x_1 = 1.5$, $x_2 = 0.8$. The highlighted path (blue → blue → green) shows the sequence of tests evaluated: $x_1 \leq 3.0$ (Yes) → $x_2 \leq 1.5$ (Yes) → predict **Class A**.

Axis-aligned splits

In most standard decision tree implementations, tests are **univariate** and **axis-aligned**: they compare *one feature at a time* to a threshold (for continuous features) or to a set of categories (for categorical features).

Entry Ticket 2

Suppose a tree uses the test "$x_3 \leq 2.5$" at the root.

1. What kind of feature is x_3 likely to be (categorical or numerical)? Why?
2. How does this test partition the feature space geometrically?

3 Learning a Decision Tree

3.1 The Recursive Induction Procedure

Decision tree learning is typically a **top-down, greedy, recursive** procedure:

1. Start with all training examples at the root node.

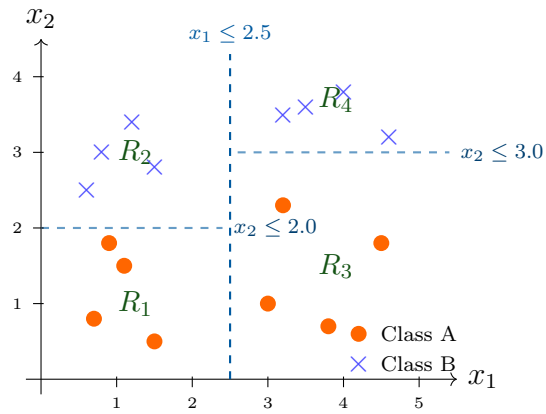

Figure 3: Axis-aligned splits partition the 2D feature space into **rectangular regions** $R_1, \ldots, R_4$. Each region corresponds to a leaf. All boundaries are parallel to the coordinate axes.

2. Choose a feature test to **split** the node into children.
3. Send each training example to the appropriate child.
4. Recurse on each child until stopping criteria are met.

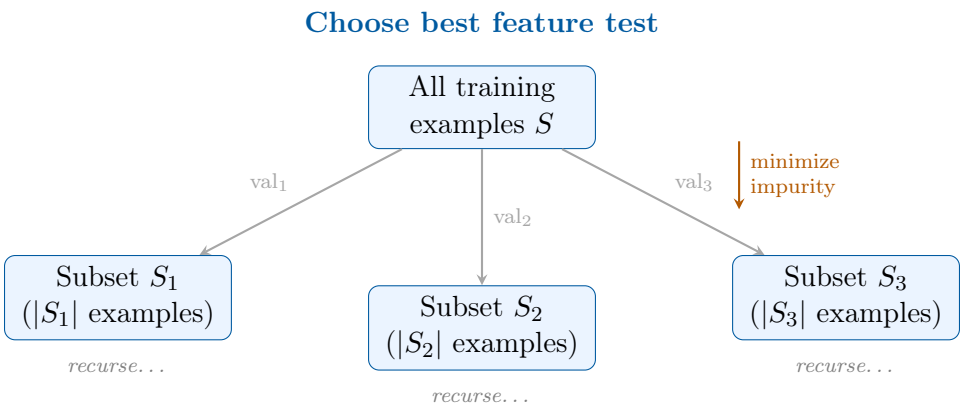

Figure 4: One step of recursive tree induction. The training set S is split into three subsets according to the values of the chosen feature. The algorithm then recurses on each subset.

Core Principle

At each node, choose the split that produces the **purest children** according to an **impurity measure**.

3.2 Stopping Criteria

Common stopping criteria include:

- The node is **pure** (all examples have the same class).
- The tree reached a **maximum depth**.
- The node has fewer than **min_samples_split** examples.

- Splitting does not improve impurity enough (a minimum gain threshold).

Greedy does not mean optimal

The greedy choice at each node does not guarantee a globally optimal tree. However, it is fast and often performs very well in practice.

Entry Ticket 3

Why might stopping early (e.g., small `max_depth`) **improve** generalization, even if the resulting tree fits the training data less accurately?

4 Decision Trees for Classification

In classification trees, each leaf predicts a class. Depending on the implementation, a leaf may store:

- the **majority class** among training examples at that leaf; and/or
- a **class distribution** (useful for probabilistic predictions).

4.1 Impurity and "Purity"

Let S be the set of training examples arriving at a node, and let the target label take values in $\{1, 2, \ldots, K\}$. Define

$$p_k = \Pr(y = k \mid \mathbf{x} \in S) \quad \text{(estimated by relative frequency in } S\text{)}.$$

An impurity function $I(S)$ should satisfy:

- $I(S) = 0$ when the node is **pure** (one class has probability 1).
- $I(S)$ is larger when the class distribution is more mixed.

4.2 Entropy and Information Gain

Entropy

The **entropy** of the class distribution at a node with example set S is

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k. \tag{1}$$

Entropy is maximal when the classes are evenly balanced (high uncertainty), and minimal (zero) when the node is pure.

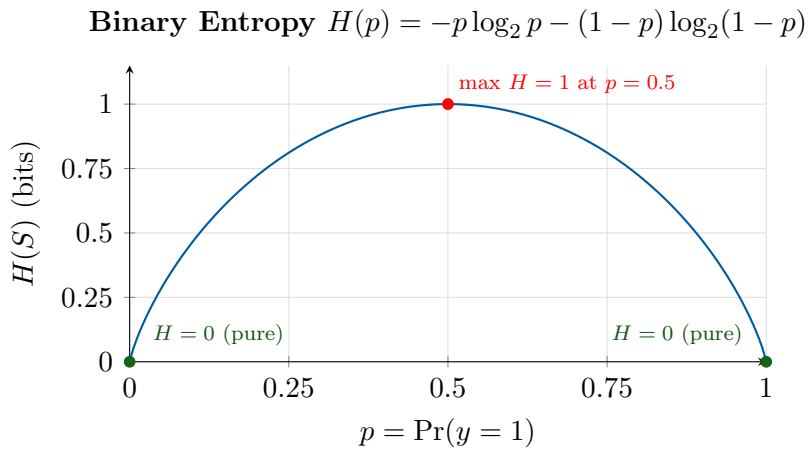

Figure 5: Binary entropy $H(p)$ as a function of $p = \Pr(y = 1)$. Entropy is zero at the extremes (pure nodes) and peaks at $p = 0.5$ (maximum uncertainty).

4.2.1 Weighted entropy after a split

Consider splitting S into disjoint subsets $S_1, \ldots, S_r$ using a feature test. The **weighted average entropy** after the split is

$$H(S \mid \text{split}) = \sum_{j=1}^{r} \frac{|S_j|}{|S|} H(S_j). \tag{2}$$

Information Gain

The **information gain** (IG) of a split is

$$IG = H(S) - H(S \mid \text{split}). \tag{3}$$

Maximizing IG is equivalent to minimizing the weighted entropy after the split.

Entry Ticket 4

Consider a binary classification node where $p = \Pr(y = 1) = 0.5$.

1. Is entropy $H(S)$ high or low? Explain intuitively.
2. If a split produces one pure child and one mixed child, how would you expect IG to behave (small or large)? Why?

4.3 Gini Impurity

Gini Impurity

The **Gini impurity** at a node with example set S is

$$I_G(S) = 1 - \sum_{k=1}^{K} p_k^2. \tag{4}$$

For a split that produces subsets $S_1, \ldots, S_r$, the **weighted Gini** is

$$G(S \mid \text{split}) = \sum_{j=1}^{r} \frac{|S_j|}{|S|} I_G(S_j). \tag{5}$$

The Gini criterion chooses the split that **minimizes** $G(S \mid \text{split})$.

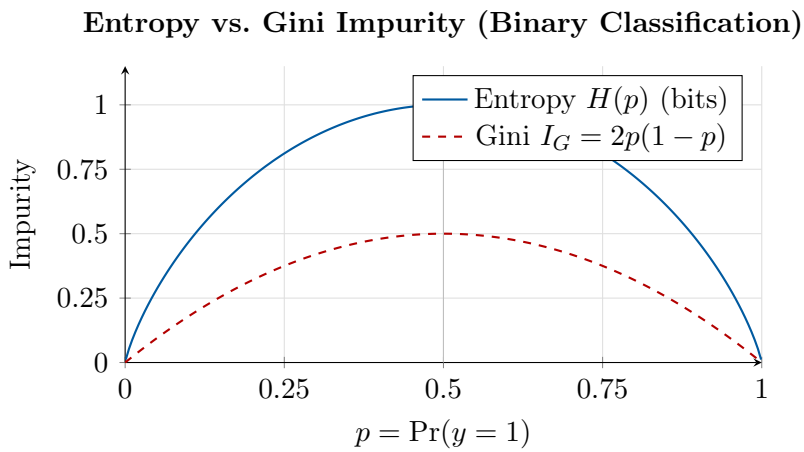

Figure 6: Comparison of entropy and Gini impurity for binary classification as a function of $p = \Pr(y = 1)$. Both measures peak at $p = 0.5$ (maximum impurity) and are zero at $p \in \{0, 1\}$ (pure nodes). They typically rank splits very similarly in practice.

Entropy vs. Gini

Entropy and Gini often rank splits similarly in practice. Both capture the idea: "prefer splits that make children as pure as possible".

Entry Ticket 5

For binary classification with $p = \Pr(y = 1)$:

1. Compute $I_G(S)$ when $p = 0.5$.
2. Compute $I_G(S)$ when $p = 0.9$.
3. Which node is "purer" according to Gini, and why?

5 A Worked Example: "Play Golf" Split Quality

This section mirrors a classic toy dataset where the target is `PlayGolf` (Yes/No) and one candidate root attribute is `Outlook` with values `Sunny`, `Overcast`, `Rainy`. The purpose is to illustrate how split criteria are computed.

5.1 Information Gain Values (Illustrative)

For this dataset (14 examples), one finds the following information gain scores:

- $IG(\texttt{Outlook}) \approx 0.247$
- $IG(\texttt{Humidity}) \approx 0.152$

- $IG(\texttt{Windy}) \approx 0.048$
- $IG(\texttt{Temp}) \approx 0.029$

Thus, `Outlook` would be selected as the root attribute under the IG criterion.

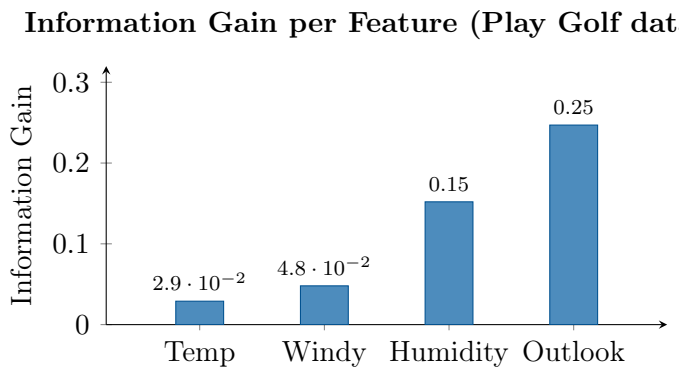

Figure 7: Information gain for each candidate feature in the Play Golf dataset. `Outlook` achieves the highest IG and is selected as the root split.

5.2 Gini Computation for `Outlook`

In the PlayGolf dataset there are 14 examples and 9 positive (`Yes`) vs. 5 negative (`No`) outcomes. Splitting on `Outlook` produces three subsets; Table 1 shows the class counts in each subset.

Outlook	**Yes**	**No**	**Total**
Sunny	2	3	5
Overcast	4	0	4
Rainy	3	2	5
All	9	5	14

Table 1: Class counts per `Outlook` value in the PlayGolf dataset.

We compute I_G for each child subset directly from the counts:

$$I_G(S_{\texttt{Sunny}}) = 1 - \left[\left(\frac{2}{5}\right)^2 + \left(\frac{3}{5}\right)^2\right] = 1 - [0.16 + 0.36] = 0.48, \tag{6}$$

$$I_G(S_{\texttt{Overcast}}) = 1 - \left[\left(\frac{4}{4}\right)^2 + \left(\frac{0}{4}\right)^2\right] = 1 - [1 + 0] = 0, \tag{7}$$

$$I_G(S_{\texttt{Rainy}}) = 1 - \left[\left(\frac{3}{5}\right)^2 + \left(\frac{2}{5}\right)^2\right] = 1 - [0.36 + 0.16] = 0.48. \tag{8}$$

The `Overcast` subset is **pure** ($I_G = 0$): every example in it is `Yes`. The other two subsets remain mixed.

Finally, the weighted Gini for the split is:

$$G(S \mid \texttt{Outlook}) = \frac{5}{14} \cdot 0.48 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.48 \approx 0.342.$$

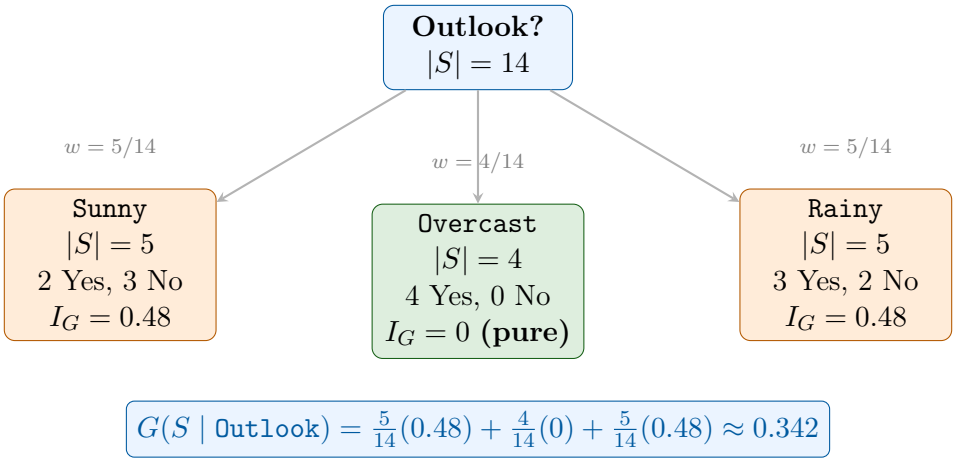

Figure 8: Gini impurity computation for the `Outlook` split. Each child node shows its class counts and resulting I_G. The `Overcast` subset is pure; the other two are still mixed. The weighted Gini of the split is ≈ 0.342.

Entry Ticket 6

Why does the weighted sum in $H(S \mid \text{split})$ or $G(S \mid \text{split})$ use factors $\frac{|S_j|}{|S|}$? What would go wrong if we simply averaged the impurities without weights?

6 Decision Trees for Regression

Decision trees can also solve regression tasks. In regression trees:

- A leaf prediction is typically the **average** of target values among training examples reaching that leaf.
- The impurity measure must handle a **continuous** target.

6.1 MSE as Node Impurity

Let D_t be the set of training examples reaching node t and let $m_t = |D_t|$. Define the mean target value at node t:

$$\overline{y}_t = \frac{1}{m_t} \sum_{i \in D_t} y^{(i)}.$$

A common impurity for regression trees is the mean squared error within the node:

$$I(t) = \text{MSE}(t) = \frac{1}{m_t} \sum_{i \in D_t} \left(y^{(i)} - \overline{y}_t\right)^2. \tag{9}$$

Given a split that divides D_t into children D_L and D_R, the split quality is measured by the **weighted** average impurity:

$$I_{\text{after}} = \frac{|D_L|}{|D_t|} I(L) + \frac{|D_R|}{|D_t|} I(R),$$

and the algorithm chooses the split that **minimizes** this value.

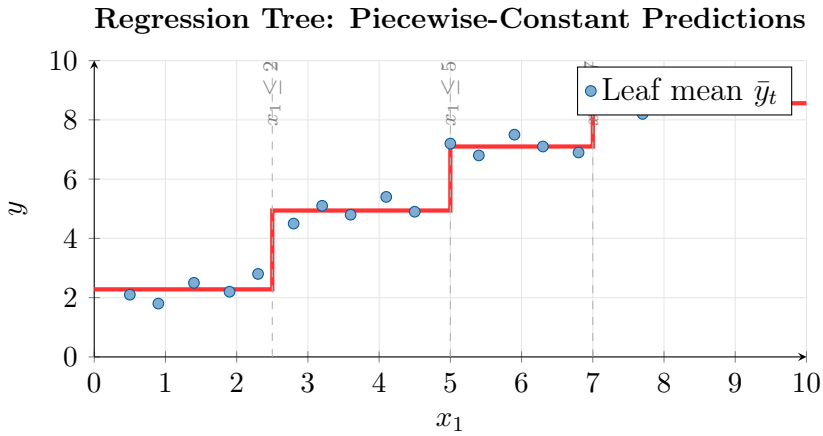

Figure 9: A regression tree partitions the input space into intervals (via axis-aligned splits) and predicts the **mean target value** $\overline{y}_t$ within each interval. The result is a piecewise-constant function.

Entry Ticket 7

In a regression tree leaf, why is predicting $\overline{y}_t$ (a constant value) a reasonable choice? Hint: think about which constant minimizes squared error over the examples in the leaf.

6.2 Overfitting in Regression Trees

If a tree grows too deep, it can create very small leaves and match noise in the training data. This is called **overfitting**. Depth control (e.g., `max_depth`) and minimum leaf sizes (e.g., `min_samples_leaf`) are key to generalization.

7 Inductive Bias of Decision Trees

Decision trees assume that the target function can be represented (or well-approximated) as a **hierarchical set of decision rules** based on feature values.

In the standard setting, this implies:

- The feature space is recursively partitioned into regions.
- For continuous features, these regions are often **rectangles** due to axis-aligned splits.
- Each region corresponds to a simple rule of the form "feature $\leq$ threshold" (or category membership).

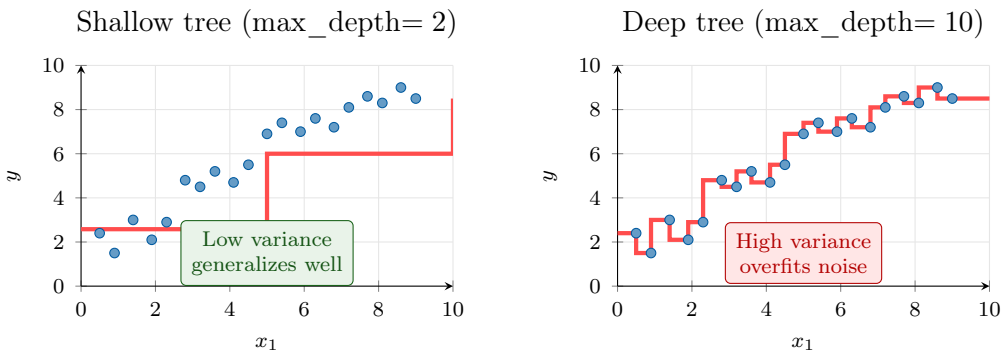

Figure 10: Effect of tree depth on regression. A **shallow tree** (left) produces a smooth, generalizable function. A **deep tree** (right) memorizes individual training points — a classic case of overfitting.

Interpretability as rules

A path from the root to a leaf can be read as an **if-then rule**:

$$\text{IF } (\text{test}_1) \ \wedge \ (\text{test}_2) \ \wedge \ \cdots \ \text{THEN predict } \hat{y}.$$

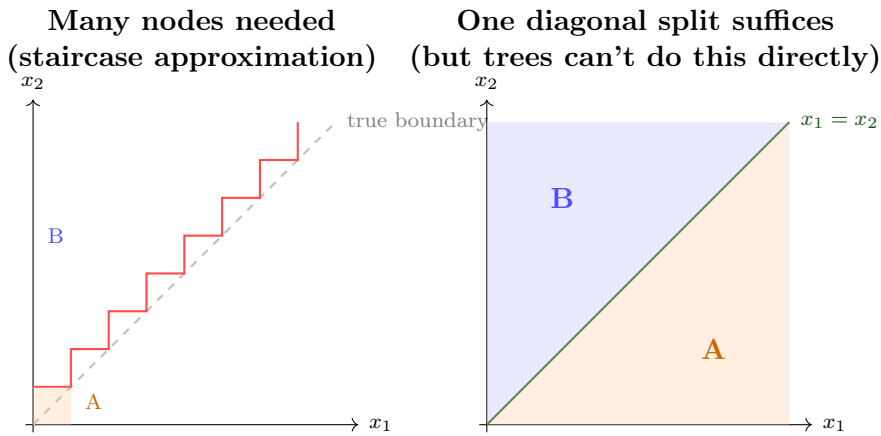

Figure 11: Axis-aligned splits struggle with **diagonal decision boundaries**. Left: a tree needs many staircase-like splits to approximate the diagonal. Right: the true boundary $x_1 = x_2$ could be captured by a single oblique split, but standard decision trees cannot do this directly.

Entry Ticket 8

Decision trees make **axis-aligned** splits by default. Describe a simple dataset pattern (in 2D) that might be hard to capture efficiently with axis-aligned splits (i.e., would require many nodes).

8 Practical Remarks

8.1 Advantages

Common advantages of decision trees include:

- They are simple to understand and interpret.
- They require little data preparation (often no normalization).
- Prediction is fast: following one root-to-leaf path.
- They handle numerical and categorical data.

8.2 Limitations

Typical limitations include:

- High variance: small data changes can yield different trees.
- Overfitting if the tree grows too deep.
- Axis-aligned splits may need many nodes for diagonal/curved boundaries.

Where trees shine

Even when a single tree is not the best performer, trees become extremely powerful as building blocks for **ensembles** (e.g., Random Forests, Gradient Boosted Trees).

9 Summary

Topic	Key idea / formula
Prediction	Follow tests from root to leaf
Classification impurity (Entropy)	$H(S) = -\sum_k p_k \log_2 p_k$
Information gain	$IG = H(S) - \sum_j \frac{\lvert S_j \rvert}{\lvert S \rvert} H(S_j)$
Classification impurity (Gini)	$I_G(S) = 1 - \sum_k p_k^2$
Regression impurity (MSE)	$I(t) = \frac{1}{m_t} \sum_{i \in D_t} (y^{(i)} - \overline{y}_t)^2$
Learning strategy	Greedy, recursive splitting with stopping criteria

Key Takeaways

1. Decision trees implement prediction as a sequence of tests on feature values.
2. Tree induction is greedy: choose splits that reduce impurity the most at each node.
3. For classification, common criteria are information gain (entropy-based) and Gini impurity.
4. For regression, a common impurity measure is node MSE (variance around the leaf mean).
5. Controlling depth and minimum leaf sizes is crucial to avoid overfitting.

A Appendix: Two Quick Sanity Checks

A.1 Gini for Binary Classification

For $K = 2$ and $p = \Pr(y = 1)$, $\Pr(y = 0) = 1 - p$:

$$I_G(S) = 1 - \left(p^2 + (1-p)^2\right) = 2p(1-p).$$

So I_G is maximal at $p = 0.5$ and minimal at $p \in \{0, 1\}$.

A.2 Why the Mean Minimizes Squared Error in a Leaf

Given values $y_1, \ldots, y_n$, the constant c that minimizes $\sum_{i=1}^{n}(y_i - c)^2$ is $c = \overline{y}$ (the sample mean). This justifies using $\overline{y}_t$ as the prediction in a regression leaf.